

Definición de requerimientos y caso de uso

Producción de un estimador del Índice de Integridad Ecosistémica a partir de
datos geográficos vectoriales y raster

Table of contents

1	Propósito	3
2	Caso de uso	3
2.1	Nombre del caso de uso	3
2.2	Descripción general	4
2.3	Actores	4
2.3.1	Actor principal	4
2.3.2	Actores secundarios	4
2.4	Disparador	4
2.5	Resultado esperado	5
3	Alcance	5
3.1	Incluye	5
3.2	No incluye	5
4	Objetivo general	5
5	Objetivos específicos	6
6	Supuestos del proceso	6
7	Requerimientos funcionales	6
7.1	RF-01. Preparación de polígonos o regiones de referencia	6
7.2	RF-02. Generación de rasters de referencia por región	6
7.3	RF-03. Producción de coberturas raster congruentes	7
7.4	RF-04. Conversión de coberturas a series numéricas congruentes	7
7.5	RF-05. Almacenamiento serializado intermedio	7
7.6	RF-06. Integración de features en una tabla maestra	8

7.7	RF-07. Preparación de datos para inferencia bayesiana	8
7.8	RF-08. Integración de predicciones del modelo	8
7.9	RF-09. Reconstrucción raster del índice final	8
7.10	RF-10. Exportación de productos cartográficos finales	8
7.11	RF-11. Trazabilidad del flujo	9
7.12	RF-12. Ejecución automatizada por reglas	9
8	Requerimientos no funcionales	9
8.1	RNF-01. Reproducibilidad	9
8.2	RNF-02. Modularidad	9
8.3	RNF-03. Escalabilidad	9
8.4	RNF-04. Paralelización	9
8.5	RNF-05. Trazabilidad de errores	9
8.6	RNF-06. Validación automatizada	10
8.7	RNF-07. Portabilidad razonable	10
8.8	RNF-08. Headless-first	10
9	Arquitectura lógica del proceso	10
9.1	Etapla 1. Geometría de referencia	10
9.2	Etapla 2. Generación de variables espaciales	10
9.3	Etapla 3. Conversión a series congruentes	10
9.4	Etapla 4. Integración tabular	11
9.5	Etapla 5. Preparación para modelado e inferencia	11
9.6	Etapla 6. Inferencia externa y reconstrucción cartográfica	11
10	Contrato mínimo de datos intermedios	11
10.1	Contrato para tablas de features	11
10.2	Contrato para tablas base canónicas	12
10.3	Contrato para rasters congruentes	12
11	Reglas de integridad del flujo	12
12	Estrategia de implementación recomendada	13
12.1	Orquestación	13
12.2	Desarrollo de scripts	13
12.3	Pruebas automatizadas	13
12.3.1	Unitarias	13
12.3.2	Integración por script	14
12.3.3	Integración de workflow	14
13	Riesgos y puntos críticos	14
13.1	Riesgo 1. Desalineación espacial	14
13.2	Riesgo 2. Dependencia implícita del orden de filas	14
13.3	Riesgo 3. Inconsistencia de nombres y columnas	14

13.4	Riesgo 4. Divergencias entre implementaciones R y Python	15
13.5	Riesgo 5. Reproyección que altera la malla útil	15
13.6	Riesgo 6. Sincronización entre máquinas y placeholders de nube	15
13.7	Riesgo 7. Metadatos locales de Snakemake	15
14	Criterios de aceptación	15
15	Estado validado actual del workflow	16
16	Hallazgos operativos de la fase de ensayos y consolidación	16
16.1	Hallazgo 1. La congruencia por píxel debe construirse solo sobre celdas válidas	16
16.1.1	Decisión adoptada	17
16.1.2	Implicación	17
16.2	Hallazgo 2. No debe materializarse la grilla completa antes del filtrado	17
16.2.1	Decisión adoptada	17
16.3	Hallazgo 3. La organización de raw/ como carpeta plana no escala bien	18
16.3.1	Decisión adoptada	18

1 Propósito

Este documento define el caso de uso, los requerimientos funcionales y no funcionales, y la arquitectura lógica de un flujo de trabajo para producir un **estimador espacial del Índice de Integridad Ecosistémica (IIE)** a partir de datos geográficos vectoriales y raster. El flujo está concebido como una tubería reproducible, escalable y preferentemente **headless-first**, con capacidad de integración en un sistema de orquestación como **Snakemake**.

El objetivo final del proceso es generar, por región de análisis, un conjunto de productos congruentes entre sí y trazables, culminando en la producción de **mapas raster del índice de integridad ecosistémica** y sus insumos tabulares intermedios.

En el estado actual del proyecto, el tramo **validado y considerado canónico** llega hasta la producción de la tabla de entrada a Netica (**bn_input.csv**). La reincorporación automática de predicciones externas de Netica al flujo geoespacial se considera aún **provisional**, pues hoy no existe garantía suficiente de congruencia fila a fila entre la tabla exportada por Python y la tabla de predicciones devuelta por Netica.

2 Caso de uso

2.1 Nombre del caso de uso

Producción de un estimador espacial del Índice de Integridad Ecosistémica

2.2 Descripción general

A partir de una colección heterogénea de insumos geográficos —capas vectoriales, coberturas raster, tablas derivadas y salidas de un modelo probabilístico o red bayesiana— se requiere construir una representación espacialmente congruente del Índice de Integridad Ecosistémica para un conjunto de regiones costeras.

El proceso debe:

1. definir una geometría de referencia común por región;
2. producir coberturas raster congruentes para múltiples variables ambientales y antrópicas;
3. convertir dichas coberturas a series tabulares congruentes por píxel;
4. ensamblar una tabla maestra de entrenamiento o inferencia;
5. usar un motor de modelado probabilístico para producir valores estimados del índice;
6. reconstruir los mapas raster finales del índice a partir de dichas predicciones.

2.3 Actores

2.3.1 Actor principal

- **Equipo técnico del proyecto:** desarrolla, ejecuta, valida y ajusta el flujo de trabajo.

2.3.2 Actores secundarios

- **Motor de inferencia bayesiana:** Netica u otro sistema equivalente que procese la tabla preparada y devuelva predicciones por observación/píxel.
- **Sistema de orquestación:** Snakemake u otro administrador de flujos que ejecute reglas, dependencias y paralelización.
- **Entorno SIG de validación:** QGIS u otra herramienta para inspección cartográfica de resultados.

2.4 Disparador

El proceso se ejecuta cuando se dispone de:

- insumos geográficos base;
- definición de regiones de análisis;
- reglas para generar variables derivadas;
- una especificación del modelo probabilístico o una salida de inferencia ya producida.

2.5 Resultado esperado

El flujo produce:

- una colección de **rasters de referencia** por región;
- una colección de **features tabulares congruentes** por región o globales;
- una **tabla final integrada** de variables por píxel;
- una **tabla de entrada a inferencia bayesiana**;
- opcionalmente, una **salida de inferencia** del índice y su reconstrucción espacial;
- un conjunto potencial de **GeoTIFF finales del Índice de Integridad Ecosistémica**.

3 Alcance

3.1 Incluye

- preparación de geometrías de referencia;
- generación de variables espaciales a partir de insumos vectoriales y raster;
- integración tabular de variables congruentes;
- preparación de tablas para modelado;
- validación estructural y pruebas operativas del flujo;
- preparación de insumos para el motor bayesiano externo.

3.2 No incluye

- diseño conceptual del índice ecológico en sí;
- definición científica de cada variable o de sus umbrales;
- captura primaria de datos de campo;
- desarrollo de interfaces gráficas como requisito principal;
- operación manual interactiva como mecanismo central del flujo;
- cierre definitivo de la reintegración automática de predicciones externas cuando no exista congruencia verificable entre tabla base y salida del motor probabilístico.

4 Objetivo general

Implementar un flujo reproducible y escalable que permita estimar y mapear el Índice de Integridad Ecosistémica a partir de múltiples fuentes geoespaciales, preservando la congruencia espacial entre todas las variables y asegurando trazabilidad desde los insumos hasta el raster final.

5 Objetivos específicos

1. Definir una **mall**a o **plantilla raster de referencia** por región de análisis.
2. Generar variables espaciales derivadas a partir de insumos geográficos heterogéneos.
3. Convertir las variables espaciales en **series tabulares congruentes por píxel**.
4. Construir una tabla final apta para entrenamiento, inferencia o exportación a motores externos.
5. Preparar una tabla canónica de entrada para inferencia bayesiana.
6. Integrar la salida del modelo probabilístico con la geometría espacial de referencia cuando exista congruencia verificable.
7. Garantizar reproducibilidad, escalabilidad y validación automatizada.

6 Supuestos del proceso

- Existe una partición territorial en regiones de análisis.
- Cada región cuenta o puede contar con un raster de referencia que fija CRS, resolución, extensión y alineación.
- Las variables derivadas pueden expresarse finalmente de forma raster o tabular por píxel.
- La congruencia espacial entre variables es un requisito central.
- Los productos intermedios se almacenan preferentemente en **Parquet** para tablas y en **GeoTIFF** para rasters.
- La salida del modelo probabilístico solo puede reincorporarse automáticamente si preserva una correspondencia inequívoca con la tabla base.
- El orden de filas y la longitud total de las tablas son parte del contrato operativo cuando interviene un motor externo sin llaves espaciales explícitas.

7 Requerimientos funcionales

7.1 RF-01. Preparación de polígonos o regiones de referencia

El sistema debe permitir preparar una colección de regiones o polígonos de análisis a partir de insumos vectoriales, incluyendo operaciones como reproyección, corrección de CRS y buffering cuando el flujo lo requiera.

7.2 RF-02. Generación de rasters de referencia por región

El sistema debe generar, para cada región, un raster de referencia que defina la malla espacial canónica del análisis, incluyendo:

- resolución;
- extensión;
- alineación;
- sistema de coordenadas;
- nodata.

7.3 RF-03. Producción de coberturas raster congruentes

El sistema debe permitir generar coberturas raster congruentes por región a partir de distintos tipos de datos de entrada, incluyendo:

- datos vectoriales rasterizados;
- datos raster reproyectados y recortados;
- interpolaciones o transformaciones espaciales;
- muestreos o distancias evaluadas sobre una base tabular canónica cuando esto resulte más estable que rederivar la malla desde un raster reproyectado.

7.4 RF-04. Conversión de coberturas a series numéricas congruentes

El sistema debe convertir las coberturas congruentes a tablas o series numéricas por píxel, preservando al menos:

- identificador regional;
- identificador de píxel;
- coordenadas x , y ;
- una o más variables temáticas.

7.5 RF-05. Almacenamiento serializado intermedio

Para las tablas intermedias y finales, se adopta **Parquet** como formato base de almacenamiento tabular, por las siguientes razones:

- mayor eficiencia de lectura y escritura;
- mejor compresión;
- mejor soporte para tablas grandes;
- compatibilidad con particionamiento por región o variable;
- mejor integración con workflows escalables.

7.6 RF-06. Integración de features en una tabla maestra

El sistema debe ensamblar una tabla maestra a partir de múltiples productos serializados congruentes, preservando la correspondencia espacial entre observaciones.

7.7 RF-07. Preparación de datos para inferencia bayesiana

El sistema debe producir una tabla de entrada apta para ser consumida por un motor de inferencia bayesiana o un proceso equivalente, incluyendo discretización y codificación cuando sea necesario.

7.8 RF-08. Integración de predicciones del modelo

El sistema debe poder leer una salida de inferencia generada por Netica o por otro motor compatible y asociarla correctamente con los píxeles de la tabla base **solo si** se cumple una congruencia verificable en orden y número de filas.

7.9 RF-09. Reconstrucción raster del índice final

El sistema debe reconstruir, por región, el raster final del Índice de Integridad Ecosistémica usando:

- la tabla base espacialmente congruente;
- las predicciones del modelo;
- el raster de referencia regional.

7.10 RF-10. Exportación de productos cartográficos finales

El sistema debe generar productos finales al menos en formato:

- GeoTIFF por región;

y opcionalmente:

- histogramas PNG;
- tablas CSV auxiliares;
- proyectos QGIS para inspección.

7.11 RF-11. Trazabilidad del flujo

El sistema debe permitir rastrear qué insumos y qué pasos generaron cada producto final o intermedio.

7.12 RF-12. Ejecución automatizada por reglas

El sistema debe permitir la ejecución automatizada de los pasos mediante reglas y dependencias explícitas, preferentemente con Snakemake.

8 Requerimientos no funcionales

8.1 RNF-01. Reproducibilidad

El flujo debe producir resultados reproducibles a partir de la misma configuración, insumos y versión de scripts.

8.2 RNF-02. Modularidad

Cada componente debe corresponder a una etapa funcional clara, con entradas y salidas explícitas.

8.3 RNF-03. Escalabilidad

El flujo debe permitir crecer en número de variables y regiones sin rediseño estructural.

8.4 RNF-04. Paralelización

Siempre que sea viable, las ramas por variable o por región deben poder ejecutarse en paralelo.

8.5 RNF-05. Trazabilidad de errores

Las fallas deben poder localizarse a nivel de regla, script, región o variable.

8.6 RNF-06. Validación automatizada

El proyecto debe contar con pruebas automatizadas que validen componentes críticos del flujo y contratos de datos.

8.7 RNF-07. Portabilidad razonable

El flujo debe ejecutarse en entornos Python reproducibles, idealmente mediante entornos controlados y dependencias explícitas.

8.8 RNF-08. Headless-first

El flujo debe funcionar sin depender de interfaces gráficas, salvo para validación o inspección posterior.

9 Arquitectura lógica del proceso

9.1 Etapa 1. Geometría de referencia

Se construyen las regiones y sus plantillas raster de referencia. Esta etapa fija la congruencia espacial del resto del proceso.

Producto principal: `ref_grid.tif` por región.

9.2 Etapa 2. Generación de variables espaciales

Se generan capas raster o equivalentes para variables derivadas a partir de datos vectoriales o raster.

Producto principal: features regionales tabulares congruentes o insumos preparados para su conversión tabular.

9.3 Etapa 3. Conversión a series congruentes

Cada variable se transforma a una tabla de observaciones por píxel con las mismas llaves espaciales.

Producto principal: archivos serializados por variable, con columnas como `regionid`, `pixid`, `x`, `y` y la variable temática.

9.4 Etapa 4. Integración tabular

Se ensamblan todas las variables en una sola tabla maestra.

Producto principal: `master_features.parquet`.

9.5 Etapa 5. Preparación para modelado e inferencia

Se prepara la tabla final para el motor bayesiano externo.

Producto principal: `bn_input.parquet` y `bn_input.csv`.

9.6 Etapa 6. Inferencia externa y reconstrucción cartográfica

Se leen predicciones del motor externo y, si existe congruencia verificable, se reconstruyen mapas raster finales por región.

Producto principal esperado: `master_features_with_ie.parquet` y GeoTIFF regionales del índice.

Estado actual: esta etapa se considera **no cerrada** mientras no se garantice correspondencia inequívoca entre `bn_input.csv` e `ie_predictions.csv`.

10 Contrato mínimo de datos intermedios

10.1 Contrato para tablas de features

Toda tabla serializada de features debe incluir como mínimo:

- `regionid`
- `pixid`
- `x`
- `y`

más una o varias columnas temáticas.

10.2 Contrato para tablas base canónicas

Cuando una feature se construya usando una **tabla base regional** para preservar alineación, esa tabla base debe cumplir el mismo contrato mínimo:

- `regionid`
- `pixid`
- `x`
- `y`

y debe representar la malla canónica que ya fue validada por el flujo.

10.3 Contrato para rasters congruentes

Todo raster congruente debe ser compatible con el raster de referencia regional en:

- CRS;
- resolución;
- `transform`;
- `shape`;
- `nodata`;
- extensión o máscara válida.

11 Reglas de integridad del flujo

1. Ninguna variable debe perder su correspondencia espacial con la plantilla regional.
2. La unión entre variables debe preservar una llave espacial consistente o una alineación posicional previamente validada.
3. La reconstrucción raster final debe usar la misma plantilla de referencia que originó la congruencia del resto del análisis.
4. Los productos intermedios deben ser suficientemente explícitos para permitir depuración y recomputación parcial.
5. Las salidas del modelo probabilístico deben poder vincularse sin ambigüedad con la tabla base.
6. Debe distinguirse explícitamente entre:
 - **clave estructural de feature**: nombre de carpeta o regla en `results/features/`;
 - **nombre temático de columna**: nombre final exportado dentro de la tabla.
7. No debe asumirse que el nombre de la carpeta de una feature y el nombre de su columna temática son idénticos.

12 Estrategia de implementación recomendada

12.1 Orquestación

Se recomienda implementar el flujo mediante **Snakemake**, organizando reglas por capas:

- `reference`
- `feature_tables`
- `training_table`
- `bayes_io`
- `final_maps`

12.2 Desarrollo de scripts

Se recomienda que cada script:

- reciba rutas parametrizadas;
- pueda ejecutarse por región cuando sea razonable;
- tenga entradas y salidas explícitas;
- no dependa de rutas hardcodeadas como única opción operativa;
- documente si usa como base un `ref_grid.tif` o una tabla base regional ya alineada.

12.3 Pruebas automatizadas

Se recomienda una batería de pruebas en **pytest** en tres niveles:

12.3.1 Unitarias

Para funciones puras:

- normalización;
- discretización;
- construcción de llaves;
- ordenamiento de regiones;
- validación de contratos;
- normalización de nombres de columnas o categorías.

12.3.2 Integración por script

Para mini-casos sintéticos:

- creación de `ref_grid`;
- rasterización simple;
- generación de `.parquet`;
- ensamblado tabular;
- alineación ligera entre features.

12.3.3 Integración de workflow

Para un DAG mínimo:

- una región;
- un subconjunto pequeño de variables;
- ensamblado final;
- preparación de `bn_input.csv`.

13 Riesgos y puntos críticos

13.1 Riesgo 1. Desalineación espacial

Si una variable no respeta la malla canónica regional, toda la integración posterior puede quedar sesgada.

13.2 Riesgo 2. Dependencia implícita del orden de filas

El flujo requiere especial cuidado cuando un motor externo devuelve predicciones por fila. Debe preservarse una correspondencia inequívoca entre observación y píxel. El problema sigue vigente en la interfaz con Netica y hoy constituye el principal límite funcional del tramo final. El README previo ya señalaba este riesgo y ahora se considera confirmado operativamente.

13.3 Riesgo 3. Inconsistencia de nombres y columnas

El ensamblado final puede fallar si no existe una convención estable de nombres para archivos y columnas.

13.4 Riesgo 4. Divergencias entre implementaciones R y Python

Algunas operaciones pueden no tener equivalencia exacta entre bibliotecas. Eso debe documentarse explícitamente y validarse empíricamente cuando haya datos disponibles.

13.5 Riesgo 5. Reproyección que altera la malla útil

En algunas variables, derivar la tabla de salida a partir de un `ref_grid` reproyectado puede cambiar el número de filas válidas. Cuando eso ocurra, conviene usar una tabla base regional ya alineada como malla canónica.

13.6 Riesgo 6. Sincronización entre máquinas y placeholders de nube

Cuando el repositorio de datos se sincroniza mediante Dropbox u otro servicio de nube, un archivo puede “existir” pero no estar materializado localmente. Esto puede romper lecturas con `pyarrow` o `pandas`, especialmente en Parquet, aunque Snakemake vea el archivo. El problema se observó en una segunda máquina Windows y se resolvió materializando localmente los archivos relevantes antes de ejecutar reglas finales.

13.7 Riesgo 7. Metadatos locales de Snakemake

Los metadatos de Snakemake viven en el proyecto local, no en el repositorio de datos. Al trabajar entre varias máquinas, es normal que existan outputs válidos pero sin metadata local asociada. Esto afecta la riqueza de `snakemake --report`, pero no invalida los archivos.

14 Criterios de aceptación

El flujo se considerará funcionalmente aceptable en su estado actual cuando cumpla al menos lo siguiente:

1. genera un raster de referencia válido por región;
2. genera variables intermedias congruentes y serializadas;
3. construye una tabla final con una fila por píxel y una columna por variable;
4. produce correctamente `master_features.parquet`;
5. produce correctamente `bn_input.parquet` y `bn_input.csv`;
6. permite reejecutar parcialmente etapas del flujo;
7. cuenta con validaciones mínimas para identificar desalineación, duplicados e inconsistencias de esquema.

La reincorporación de inferencia externa y la reconstrucción de mapas finales se considera aún **provisional** y no forma parte del criterio de aceptación cerrado mientras no exista correspondencia verificable con la salida de Netica.

15 Estado validado actual del workflow

En su estado actual, el workflow ya integra de forma congruente al menos las siguientes familias de variables:

- tasa_erosion
- corales
- tipo_costa
- zvh
- velocidad_del_viento
- estructuras_costeras
- spp_invasoras
- pasto_marino
- batimetria
- madmex_uso_suelo
- manglares
- movimiento_dunas

En una corrida validada con dos regiones de prueba, el ensamblado produjo `master_features.parquet` con **546223 filas** y **20 columnas**, confirmando que las nuevas variables se integran sin romper la alineación regional cuando respetan la malla canónica del workflow. :contentReferenceaicate:2

16 Hallazgos operativos de la fase de ensayos y consolidación

La fase de ensayos sobre laptop Windows y la posterior validación en una segunda máquina permitieron validar la arquitectura general del workflow, detectar defectos lógicos importantes en la traducción inicial de scripts R a Python y ajustar decisiones operativas relevantes para su ejecución estable.

16.1 Hallazgo 1. La congruencia por píxel debe construirse solo sobre celdas válidas

El principal defecto lógico detectado en la traducción inicial de algunos scripts Python fue que, después de reprojectar el `ref_grid` regional, se estaban convirtiendo **todas las celdas** de la

grilla reproyectada a observaciones tabulares, incluyendo celdas sin datos o fuera de la máscara útil.

Eso provocó una inflación artificial severa en las tablas de features.

16.1.1 Decisión adoptada

Los scripts de features deben:

1. reproyectar el raster de referencia cuando aplique;
2. identificar primero las celdas válidas;
3. calcular coordenadas x , y solo para esas celdas válidas;
4. exportar únicamente esas observaciones.

16.1.2 Implicación

La tabla tabular congruente por píxel no debe entenderse como “todas las celdas del raster reproyectado”, sino como “todas las celdas válidas de la malla útil regional”.

16.2 Hallazgo 2. No debe materializarse la grilla completa antes del filtrado

En regiones grandes, incluso el paso intermedio de construir coordenadas para toda la grilla reproyectada resultó inviable por memoria.

16.2.1 Decisión adoptada

Los scripts de features deben evitar:

- creación de `meshgrid` completo;
- transformación a coordenadas para toda la matriz;
- filtrado posterior.

En su lugar, deben trabajar así:

1. detectar índices de celdas válidas;
2. generar coordenadas solo para esas posiciones;
3. construir la tabla final directamente a partir de ese subconjunto.

16.3 Hallazgo 3. La organización de raw/ como carpeta plana no escala bien

Durante los ensayos operativos se comprobó que una carpeta **raw/** plana se vuelve rápidamente confusa y poco manejable.

16.3.1 Decisión adoptada

La organización canónica de insumos se movió a una estructura temática bajo **varsIni/**, sustituyendo el uso previo de **raw/** como convención principal. La estructura actual recomendada es del tipo:

““text varsIni/ batimetria/ coastal_regions/ corals/ dunes_cost_2014/ dunes_inegi/ erosion/ estructuras/ madmex/ manglares/ pastos_marinos/ plantas_snib/ velocidad_del_viento/ results/ reference/ features/ training/ final_maps/ external/ netica/